How far a language model can improve translation results

NLP final project: Noisy-channel approach for IT→EN translation

Mauro Venticinque

The task

- **Input:** a parallel corpus (Europarl IT–EN), nothing else.
- **Output:** an Italian→English translator, as close as possible to off-the-shelf models.

Key idea

Noisy-channel: Don't model translation directly, factor it using Bayes:

$$\hat{e} = \arg\max_{e} P(e \mid f) = \arg\max_{e} \underbrace{P(e)}_{\text{fluency}} \underbrace{P(f \mid e)}_{\text{fidelity}}$$

Thanks to factorization we can split the task in 2:

1. One model judges *meaning* of translation (Translation Model)
2. The other judges *correctness* in english (Language Model on target language)

The idea of this project is based on fixing on element, the translation model, and change LM.

Notation: the alignment problem

- $f = f_1 \dots f_m$ = Italian sentence (m words), $e = e_1 \dots e_l$ = English sentence (l words).
- **Alignment** $a = a_1 \dots a_m$: each Italian word f_j is "generated" by one English word e_{a_j}, with $a_j \in \{0, 1, \dots, l\}$ — position 0 is **NULL** (spurious Italian word: clitics, articles).
- The alignments are latent: **never observed.** We marginalize over them:

$$P(f \mid e) = \sum_a P(f, a \mid e)$$

The whole concept of statistical MT is the choice of $P(a \mid \dots)$

IBM Model 1: lexical translation only

Generative story: given e, produce f:

1. Choose length m (uniform, ignorable).
2. For each $j = 1..m$: pick a_j **uniformly** from $\{0..l\}$.
3. Emit $f_j \sim t(\cdot \mid e_{a_j})$.

$$P(f, a \mid e) = \frac{\varepsilon}{(l+1)^m} \prod_{j=1}^{m} t(f_j \mid e_{a_j})$$

- EM on this model is **concave** (unique optimum) and yields exactly the lexical table t we need.
- But the alignment prior is flat: **word order carries zero information**.

IBM 1 worked example: order is invisible

Take e = “the black cat”, f = “il gatto nero” ($m = l = 3$):

- Correct alignment $a = (1, 3, 2)$ and nonsense $a = (1, 1, 1)$ both have prior $\frac{1}{(3+1)^3} = \frac{1}{64}$.
- Only the lexical term t can prefer the correct one, and for rare words t is estimated from a handful of counts.

Consequence

IBM 1 learns *which words translate each other* but learns *nothing about where they sit*. This is called reordering problem

IBM 1 remains the best initializer for other SMT models, the t table is our translation table but as an alignment model it is blind

IBM Model 2: add distortion

Keep the lexical term, replace the flat prior with an **alignment (distortion) distribution**:

$$P(f, a \mid e) = \varepsilon \prod_{j=1}^{m} t(f_j \mid e_{a_j})\; d(a_j \mid j, l, m)$$

- $d(a_j \mid j, l, m)$ = probability that the j-th Italian word comes from English position a_j, **given the two sentence lengths**.
- Now the correct alignment of *"il gatto nero"* beats the degenerate one by a factor of $d(1|1)\, d(3|2)\, d(2|3) \gg d(1|1)\, d(1|2)\, d(1|3)$.
- Reordering enters the model but through **absolute positions**.

IBM 2's flaw: absolute positions don't travel

$d(j \mid i, l, m)$ is a lookup table indexed by **where** in the sentence:

- **Sparsity:** for each pair of lengths (l, m) you must estimate an $l \times m$ table from counts of *exactly* those positions. A swap seen at positions (2,3) of a 5-word sentence says nothing about positions (12,13) of a 15-word sentence.
- **Length-dependence:** the same linguistic reordering gets a different probability in every sentence it appears in.
- **No locality:** position 7 can align to position 2 with the same freedom as to position 6 — there is no "neighbors stay neighbors" bias.

IBM Model 3: fertility — one-to-many, explicitly

Fertility φ_i = number of Italian words that English word e_i generates, $\varphi_i \sim n(\cdot \mid e_i)$.

$$P(f, a, \varphi \mid e) = \prod_{i=1}^{l} n(\varphi_i \mid e_i) \cdot \{\text{distortion}\} \cdot \prod_{j=1}^{m} t(f_j \mid e_{a_j})$$

- Example: *"l' ho fatto"* → *"I did it"*: $a = (\text{I}, \text{did}, \text{did})$ — **did** has fertility 2 (covers *ho fatto*); the clitic *l'* aligns to *I*.
- Fertility is what makes **insertions and deletions** a feature rather than accidents of t.
- Cost: Model 3's exact EM is **computationally brutal** for large enough corpora.

HMM alignment: the Markov chain that sees locality

Scan the Italian sentence left to right; let the alignment be a **Markov chain over English positions**, with transitions driven by the **relative jump** $\Delta_j = a_j - a_{j-1}$:

$$P(a \mid e) = \prod_{j=2}^{m} p(a_j \mid a_{j-1}), \qquad P(f, a \mid e) = \underbrace{\prod_{j} p(a_j \mid a_{j-1})}_{\text{reordering}} \cdot \underbrace{\prod_{j} t(f_j \mid e_{a_j})}_{\text{lexicon}}$$

- Transition p depends only on the **jump size** ~5 parameters (`stay`, `±1`, `±2`), estimated from *all* sentence pairs at once.
- Locality is forced by the structure of the transition matrix.
- Optimized bayesan samplers make the estimations faster than IBM's EM.

The three models, side by side

	IBM 1	IBM 2	HMM (eflomal)
alignment prior	uniform $1/(l+1)$	$d(a_j \mid j, l, m)$ absolute	$p(\Delta)$ relative jump
parameters for order	none	$l \times m$ per length pair	~5 values, shared globally
sees locality?	no	weakly	yes, built-in
fertility (1-to-many)?	no	no	yes (model-3 style)
inference	trivial EM	EM	forward–backward, still fast

eflomal: the engine

What eflomal actually fits

- IBM 1 initialization → **HMM with relative-jump transitions + fertility model** (model-3-style generative process), trained with **Gibbs Sampling + hillclimbing** converges in minutes on 200k pairs, where exact model-3 EM is intractable.

- Run in **both directions** (IT→EN and EN→IT).
- **Intersect** the two link sets: a link survives only if both directions agree → high precision at the price of some recall (we can afford it: tables get pruned anyway).

Why we need *two* translation tables

il presidente interrompe l' oratore → the president cut off the speaker

IT→EN links (forward)

`0-0 1-1 2-2 3-3 4-5`

links *l'* to *off*; *cut off* only half covered

EN→IT links (reverse)

`0-0 1-1 2-2 2-3 3-4 4-5`

links *l'* to *the*; *interrompe* covers *cut off*

Intersection `0-0 1-1 2-2 4-5` = links both directions agree on → high precision. The clitic *l'* (3-3 vs 3-4) and *off* (2-3) disagree or appear once, so they drop out; unaligned words go to the deletion entry

Italian → English	forward	reverse	kept
il → the	✓	✓	✓
presidente → president	✓	✓	✓
interrompe → cut	✓	✓	✓
interrompe → off	–	✓	–
l' → off	✓	–	–
l' → the	–	✓	–
oratore → speaker	✓	✓	✓

The decoder score

LOG-LINEAR SCORE OF A HYPOTHESIS

$$\text{score} = w_{lm} \log P_{LM}(e \mid \text{ctx}) + w_{ef} \log P(e \mid f) + w_{fe} \log P(f \mid e) + \beta - \alpha \, |j - j_{prev} - 1|$$

- Each step covers **one source position**: emit its translation or **delete** it; beam search keeps the best partial hypotheses.
- LM = fluency and word order; tables = meaning; β = word bonus; α = distortion penalty (a cheap stand-in for a real reordering model).
- By construction: **no insertions, no phrases** — the LM only *chooses and reorders* proposed words.

How far *can* we go?

- **opus-mt (Helsinki-NLP)**: Marian transformer trained on OPUS (includes Europarl — same domain, far more data).
- Not a fair competitor: the **ceiling** that shows what the task allows.

	`val_short`	`val_df`
opus-mt BLEU	**40.5**	**32.9**
opus-mt chrF	59.0	57.7
time/sent	15 ms	77 ms

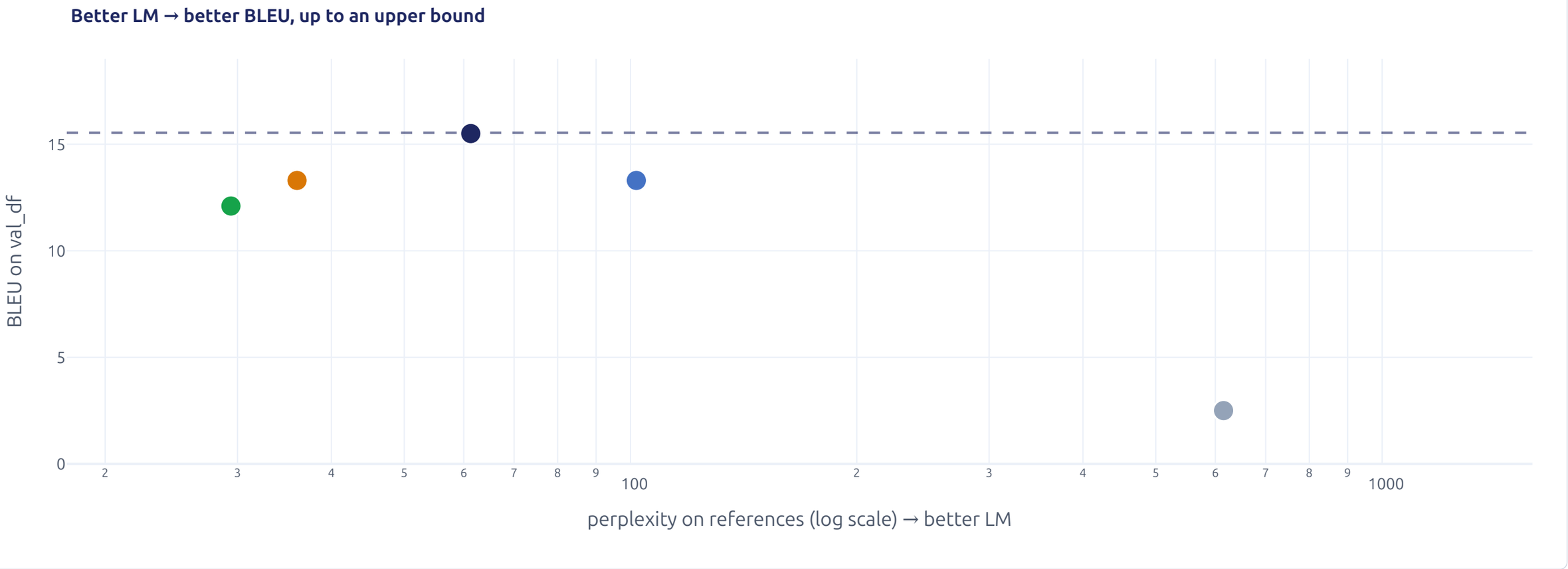
Our question: starting from raw parallel text, how close can noisy-channel decoding get?

Plugging in language models: word-level results

LM	PPL ↓	BLEU `val_df`	BLEU `val_short`	ms/sent
unigram (KN)	615.3	2.5	9.8	724
bigram (KN)	101.8	13.3	21.7	561
trigram (KN)	61.3	**15.5**	22.0	834
LSTM (trained)	**29.4**	12.1	19.7	1026
Gemma-3-1B (pretrained)	36.0	13.3	17.0	20833

- BLEU tracks LM quality until **bigram → trigram**, then **stalls**.
- LSTM and Gemma: 2–3× lower perplexity, *worse* BLEU — the LM is no longer the bottleneck.

The upper bound in one picture



The vocabulary problem

- Word-level vocab is **tens of thousands** of types on 200k pairs.
- Count-based tables → every rare word has **no or one noisy entry**.
- Rare/unseen source words **slip through**: copied unchanged, stranded at the end of the sentence.

OOV slip-through

…converremo tutti che… → *“… essential culture arts **converremo**”*
*“…area discussed **primeggiare**”*
*“…with this **disarm**”*

The LM keeps improving, but the words it would need are not in the candidate table: sparsity of $P(e|f)$, is the limit

Fix attempt: SentencePiece tokenization

- Pretrained Marian SentencePiece models → shared subword inventory, **no OOV by construction**.
- Tables and decoder operate on **pieces**; hypotheses detokenized back to words for BLEU.

Setup	BLEU `val_df`	BLEU `val_short`
trigram, words	**15.5**	22.0
trigram, pieces	12.9	22.4
LSTM, words	12.1	19.7
LSTM, pieces	14.0	**22.9**
opus-mt (pieces, trained)	32.9	40.5

- Helps the **neural** LM (+2–3 BLEU); **hurts** the count-based n-gram (finer units → sparser counts).

Fix attempt: Larger Beam

- A thight beam (10 was used) might miss better scoring translations, does a wider search?
- Results refer to trigram on `val_df` tokenized using SentencePiece

beam / candidates per word	BLEU	chrF	ms/sentence
10 / 5	12.9	47.1	380
20 / 10	13.8	47.5	1,697
50 / 10	14.4	47.9	4,247

Search error caused by small beam is real but small copared to the ~18 BLEU gap to opus-mt. The main limit is the translation model

What about a full model instead?

- Seq2seq from scratch: **BiLSTM encoder + Luong attention**, 512-d embeddings/hidden, tied weights.
- Same 200k pairs, ≤ 50 epochs, beam 5.

Seq2Seq scratch

BLEU **10.3** · chrF 31.7

len-ratio **0.50** → truncated outputs

Why it loses: from-scratch training requires

1. **Data** — 200k pairs is small for end-to-end MT;
2. **Compute** — LSTMs converge slowly; ≤ 50 epochs is not enough;
3. **Capacity** — one 512-d state must encode two languages, alignment and grammar **jointly**.

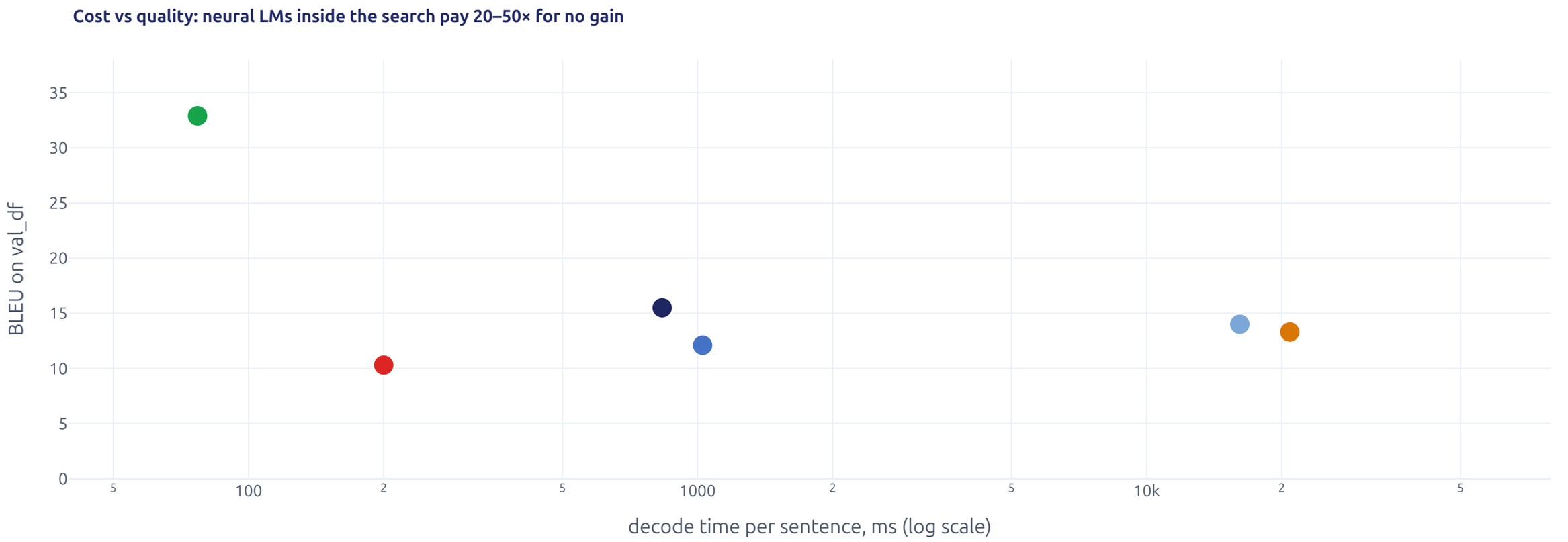
Factorized systems simplify the task: the aligner solves fidelity, the LM solves fluency; each with a simpler model

The full comparison

System	BLEU `val_df`	chrF	len-ratio	ms/sent
noisy channel + unigram	2.5	43.5	0.97	724
noisy channel + bigram	13.3	49.8	0.97	561
noisy channel + trigram	**15.5**	50.4	0.97	834
noisy channel + LSTM	12.1	49.0	0.97	1026
noisy channel + Gemma-3-1B	13.3	54.4	0.91	20833
seq2seq from scratch	10.3	31.7	0.50	~200
opus-mt (ceiling)	**32.9**	**57.7**	0.90	77

chrF gap (50 vs 58) < BLEU gap (15.5 vs 32.9): we get characters right, miss word choice and insertions → the gap is the word-level decoder, not the aligner

Cost vs quality



Scoring a 1B LM inside the beam costs 20–50× and buys nothing — use strong LMs to rerank n-best lists, not to score every hypothesis

Takeaways

- The noisy channel decomposes MT into **meaning + fluency**; HMM+fertility alignments make both cheap to estimate from 200k pairs.
- Better LMs help **until word order is solved**; then the bound is the translation model and decoder structure.
- Vocabulary sparsity, is the word-level limit → subwords help neural LMs but hurt count models.
- Remaining gap to opus-mt = bottleneck with translation model.

Thanks for listening!